

Supplementary Material (Replication Package Document) for “JDeBench: Benchmarking Coding Agents on Real-World JavaScript Deobfuscation”

ANONYMOUS AUTHOR(S)

This document records the configurations, measurement definitions, and supporting analyses underlying the main paper. Its organization follows the evaluation workflow: Sections S1–S2 specify the two protection conditions and their archived builds; Section S3 describes corpus construction and the dual-view sandbox; Section S4 defines the metrics and statistical procedures; Section S5 records the evaluated systems, inferential details, and generation-stability study; Section S6 documents the readability judge and its repeat stability; Section S7 examines hidden-identifier recovery and input-length sensitivity; and Section S8 explains cost accounting and coverage. Unless stated otherwise, JDeBench summaries use the fixed 93-module cohort from 33 projects, and JsDeObsBench summaries use a separate fixed 93-program cohort. Missing values are never imputed or scored as zero.

S1 Source-Level Obfuscation Configuration

The JSO/full condition uses javascript-obfuscator 5.5.0. The base seed is 20260809, and subject i uses seed $20260809 + i$. Global and property renaming remain disabled so that developer workloads can call the original interface. Options not shown retain the package defaults.

```
{
  compact: true, target: "node",
  renameGlobals: false, renameProperties: false,
  sourceMap: false, simplify: true,
  stringArray: true, stringArrayThreshold: 1.0,
  identifierNamesGenerator: "hexadecimal",
  stringArrayEncoding: ["rc4"],
  stringArrayRotate: true, stringArrayShuffle: true,
  splitStrings: true, splitStringsChunkLength: 8,
  stringArrayWrappersCount: 2,
  stringArrayWrappersType: "function",
  stringArrayCallsTransform: true,
  controlFlowFlattening: true,
  controlFlowFlatteningThreshold: 1.0,
  deadCodeInjection: true,
  deadCodeInjectionThreshold: 0.6,
  selfDefending: true, debugProtection: true,
  numbersToExpressions: true, transformObjectKeys: true
}
```

S2 VM Build Implementation and Archive

VM builds use the hosted Obfuscator.io service through its vmObfuscation option [5, 8]. The service replaces selected function bodies with custom bytecode executed by an embedded interpreter while retaining the surrounding JavaScript module [4]. Generating new builds requires a service credential; re-scoring the archive does not [6].

All 93 VM inputs evaluated in the main study are included in the replication package. They were built between 11 and 28 September 2026 with the service’s vmObfuscation option enabled and its remaining options at their defaults. The retained manifests identify the subject, original bundle, protected output, size, and available build metadata, but no hosted-service version identifier is available in either the manifests or the generated artifacts. The release therefore fixes the evaluated condition through the archived inputs and records the configuration; regeneration may reflect

Table S1. Final JADeBench cohort used by all main analyses.

Property	Value
Modules / projects	93 / 33
Domains: client / network / data / platform	13 / 18 / 34 / 28
Root LOC: Q1 / median / Q3	111 / 213 / 408
Root LOC range	42–3,768
Statement coverage: Q1 / median / Q3	85.8% / 97.4% / 100%
Branch coverage: Q1 / median / Q3	77.9% / 92.3% / 100%
At least two first-party modules	54/93 (58.1%)
Asynchronous control flow	51/93 (54.8%)
Licenses: MIT / ISC / BSD-3-Clause	86 / 6 / 1

subsequent changes to the hosted service. The service skips functions that contain direct eval or dynamic Function construction [3]. Settings for separate self-defending and debug-protection features are also incomplete [7, 9]. RQ3 therefore compares two complete protection configurations rather than isolating virtualization. Redistribution remains subject to the service terms.

S3 Corpus and Dual-View Sandbox Details

S3.1 Construction and Final Cohort

Repository screening requires a permissive license, recent maintenance, bounded size, and a supported native test environment. Extraction bundles first-party dependencies, leaves third-party packages external, and maps modules to developer tests. The realism screen requires two of three properties: at least three host or third-party API call sites, at least two inlined first-party modules, and at least 30% domain-bearing identifiers. Near duplicates use Jaccard similarity over normalized five-token shingles; no pair among the 265 screened modules exceeds 0.466, so the deduplication stage rejects nothing at any threshold between 0.7 and 0.9. Final admission requires a loadable bundle, a nonempty passing workload that exercises the root module, and both protected builds passing that workload in the evaluation sandbox; Table 1 of the main paper records every stage’s rejections.

The final cohort contains 93 modules from 33 projects. Table S1 is recomputed from its fixed manifest. The four largest projects contribute 30/93 modules, motivating project-clustered intervals.

S3.2 JsDeObsBench Cohort for RQ1

We follow JsDeObsBench’s upstream eligibility rules: retain one successful JavaScript submission per non-repetitive CodeNet problem and exclude programs longer than 25,000 characters [1]. At the pinned commit, the union across its eleven obfuscation configurations contains 1,285 programs. The RQ1 cohort is a fixed subset of 93 of these programs, matched in count to the 93 JADeBench modules and frozen before any result was analyzed; each program keeps the obfuscation configuration under which JsDeObsBench archived it, and its archived input–output cases serve as the workload. The subset is not a probability sample, so RQ1 is reported as a descriptive cross-cohort comparison without multiplicity-adjusted tests. The program list, with each program’s configuration, is part of the replication package.

S3.3 Public and Private Views

The public view contains the protected module, declared runtime dependencies, a generated task file, and two harness entry points. `execute.mjs` performs bounded calls through the module interface;

debug.mjs exposes bounded breakpoints, scope values, and traces. It excludes the original module, developer workload, expected values, coverage data, identifying metadata, and links into the private oracle. The builder checks this invariant after construction and the validator repeats it before admission.

The private view retains the project layout and developer workload. After a system produces answer.js, the scorer moves that candidate one way into the private view. Seeded clocks and random values and replayed network responses reduce nondeterminism. The common instruction requires one complete replacement preserving format, exports, behavior, exceptions, and host interactions. Built-in web tools are disabled, although shell-level network access is not blocked.

The fixed JADeBench archive contains 930 agent-run records (five agents, 93 subjects, and two protection conditions), each of which names a relative transcript path. The corresponding transcript bodies are not present in the analysis archive. Consequently, network-command and repository-mention counts cannot be reconstructed reliably from the retained evidence; the main paper treats external retrieval as an unmeasured threat rather than reporting inferred zero counts.

S4 Metric and Statistical Implementation

S4.1 Correctness and Similarity

Syntax correctness requires a nonempty candidate accepted by the language-aware parser and Node syntax check and exposing the original interface. Execution correctness requires successful termination, no failed tests, and the same positive test count as the qualified reference. Timeouts, zero-test runs, partial passes, missing candidates, and crashes score zero. For JsDeObsBench, every archived input-output case must match after trimming surrounding whitespace.

CodeBLEU-R is a recovery-aware adaptation of CodeBLEU [10]. It equally weights JavaScript-token BLEU, candidate-precision weighted n-gram matching, syntax-subtree F1, and normalized-data-flow F1. Multiplicity-clipped weighted matches use the candidate count as denominator, and structural components use $F_1 = 2M/(|R| + |C|)$ so extra scaffolding lowers the score. If neither side exposes data flow, that component is omitted and the other weights are renormalized. This definition is not numerically comparable with standard CodeBLEU.

ROUGE-L is whole-module longest-common-subsequence F1 [2]. For candidate tokens C , reference tokens R , and LCS length ℓ ,

$$\text{ROUGE-L}(C, R) = \frac{2\ell}{|C| + |R|}. \quad (1)$$

Both metrics use tree-sitter JavaScript leaf tokens, exclude comments and whitespace, and retain identifiers, literals, punctuation, and case. Empty or syntactically invalid candidates receive zero; execution failures remain in unconditional means.

For the identity baseline, we score each of the 93 canonical protected inputs unchanged against its original module with the same CodeBLEU-R and ROUGE-L implementations. The source-level means are 0.044 and 0.105, respectively; the VM means are 0.016 and 0.041. All 186 original-protected pairs parse without errors. To characterize name exposure, we also take the fraction of distinct binding names declared in each original that reappear as declared names in its protected build, then average these fractions across modules: 29.9% under source-level protection and 29.8% under VM protection. This name-overlap measure does not establish that a matching name denotes the same binding. Per-module scores and the recomputation script accompany the analysis artifact.

S4.2 Inference

Paired binary outcomes use two-sided exact McNemar tests. Holm correction is applied within the prespecified RQ2 and RQ3 families. RQ2 contains ten agent-versus-static-GPT-sol comparisons plus

Table S2. Evaluated systems and recorded configurations.

System	Class	Model	Version	Principal limit
webcrack	Traditional	none	2.16.0	300 s/build
JSimplifier	Fixed pipeline	LLM rename + webcrack	1.0.0	archived limit
Static GPT-5.6-sol	Static LLM	GPT-5.6-sol	API	32k output
Static GLM-5.2	Static LLM	GLM-5.2	API	131k output
Static DeepSeek-V4-Pro-0813	Static LLM	DeepSeek-V4-Pro-0813	API	archived API limit
Static Kimi-K2.5	Static LLM	Kimi-K2.5	API	archived API limit
OpenHands	Agent	GPT-5.6-sol	1.16.0	1,800 s; 120 s shell
Claude Code	Agent	GPT-5.6-sol	2.1.20	1,800 s; 120 s shell
OpenCode	Agent	GPT-5.6-sol	1.3.13	1,800 s; 64 steps
Kimi Code	Agent	GPT-5.6-sol	0.42.0	1,800 s; 64 steps
Codex	Agent	GPT-5.6-sol	0.131.0	1,800 s; 120 s shell

OpenHands–Codex under JSO/full and OpenHands–OpenCode under VM. RQ3 contains eleven within-system JSO/full–VM comparisons.

Percentile bootstrap intervals sample JsDeObsBench programs individually and JADeBench projects as clusters, retaining all modules of a sampled project. Principal execution analyses use 100,000 draws with seed 20260929; RQ1 similarity intervals use 20,000 draws with the same units. These intervals do not turn the cross-cohort RQ1 contrast into a causal corpus effect.

The frozen prediction tables contain one retained candidate per system–subject–condition cell and no duplicate scored cells. Runs without an improved candidate retain their checkpoint or protected-input fallback rather than being removed or rerun after scoring.

S5 Systems and Statistical Details

S5.1 Recorded Configurations

The unit is the delivered system, including scaffold, prompt, tools, model route, and limit. Table S2 reports archived configurations; aliases are not retroactively resolved to snapshots.

All five coding agents use a wall-clock limit of 1,800 seconds per run. Claude Code, Codex, and OpenHands additionally limit each shell command to 120 seconds, while Kimi Code and OpenCode permit at most 64 steps per run. These limits define the evaluated complete systems, and built-in web tools are disabled for every agent.

S5.2 RQ1

Execution passes on JsDeObsBench versus JADeBench JSO/full are: static GPT-sol 74 versus 17, static GLM-5.2 21 versus five, OpenHands 86 versus 78, Claude Code 80 versus 58, OpenCode 87 versus 59, Kimi Code 71 versus 47, and Codex 82 versus 54. The populations are independent. For Codex versus static GPT-sol, discordances are 16 versus eight on JsDeObsBench ($p = 0.152$) and 43 versus six on JADeBench ($p < 10^{-6}$). The 31.2-point difference between their execution gaps has a descriptive bootstrap interval of 12.3–50.8 points and combines differences in programs, workloads, and protection configurations; it is an observed cross-setting contrast rather than a causal corpus estimate.

S5.3 RQ2 and RQ3

The complete descriptive results appear once in Table 2 of the main paper; the supplementary material records the paired inferential details without duplicating that table. Binary outcomes use all 93 modules, and similarity means retain failed outputs.

Table S3. RQ2 execution differences with project-clustered percentile 95% bootstrap intervals. Values are percentage points; positive values favor the first system.

Protection	Contrast	Difference [95% interval]
JSO/full	OpenHands – Static GPT-sol	65.6 [55.0, 77.1]
	Claude Code – Static GPT-sol	44.1 [28.6, 61.3]
	OpenCode – Static GPT-sol	45.2 [31.3, 57.9]
	Kimi Code – Static GPT-sol	32.3 [14.2, 49.4]
	Codex – Static GPT-sol	39.8 [24.1, 56.5]
	OpenHands – Kimi Code	33.3 [20.0, 47.7]
VM	OpenHands – Static GPT-sol	54.8 [43.9, 64.3]
	Claude Code – Static GPT-sol	18.3 [10.1, 26.4]
	OpenCode – Static GPT-sol	22.6 [15.2, 29.4]
	Kimi Code – Static GPT-sol	22.6 [14.3, 30.4]
	Codex – Static GPT-sol	23.7 [15.7, 32.3]

All ten agent-versus-static-GPT-sol RQ2 comparisons survive Holm correction. OpenHands also exceeds Codex under JSO/full (28 versus four discordances; adjusted $p = 3.05 \times 10^{-5}$). OpenHands exceeds OpenCode under VM by 30 modules (54 versus 24 passes); with at most 24 discordant modules in OpenCode’s favor, the two-sided exact McNemar p is at most 1.0×10^{-3} for any split of the discordant modules, which survives the twelve-way Holm correction (worst-case adjusted $p < 0.02$). In RQ3, the protection-related execution decline survives Holm correction for static GPT-sol and all five agents; changes for the other static LLMs and traditional pipelines do not. These tests compare complete systems and configurations.

Table S3 reports the project-clustered intervals corresponding to the agent–static contrasts and to the descriptive largest agent spread under JSO/full. All intervals exclude zero.

S5.4 Targeted Generation-Stability Study

We assessed generation sensitivity for OpenHands and Codex on a fixed sample of 20 JSO/full modules from 20 projects. A fixed seed selected seven small, six medium, and seven large protected inputs. To expose both stable and discordant cases, selection was stratified by the two systems’ archived pass/fail records: seven modules on which both passed, seven OpenHands-only passes, three Codex-only passes, and three joint failures. This deliberately enriched sample estimates repeat stability; its raw pass rates are not estimates of the 93-module population rates.

For each system, we combine the archived main-paper output (A) with two independently generated outputs (R1 and R2). R1 and R2 record the same openai/gpt-5.6-sol route, use the same hashed protected build for each subject, and contain no input truncations. The archived run contains mixed route identifiers, including three gpt-sol-latest OpenHands records. Comparisons involving A can therefore include route or configuration drift in addition to stochastic generation; R1–R2 is the controlled fixed-route repeat comparison.

Table S4. Generation stability on the fixed 20-module JSO/full sample. Each slash-separated cell reports A/R1/R2. Execution entries are complete-workload pass counts; CodeBLEU-R and ROUGE-L are unconditional means. Agreement compares the two fixed-route repeats, and Changed counts subjects whose execution outcome differs at least once across all three observations.

System	Execution	CodeBLEU-R	ROUGE-L	R1-R2 agree	Changed
OpenHands	12/18/18	.510/.514/.510	.692/.655/.652	16/20	10/20
Codex	8/15/17	.450/.493/.494	.602/.669/.646	16/20	11/20

The two fixed-route repeats have identical OpenHands pass counts and differ by two Codex passes, but each system changes outcome on four individual modules between R1 and R2. Their mean CodeBLEU-R differs by 0.004 for OpenHands and 0.001 for Codex; mean ROUGE-L differs by 0.003 and 0.022, respectively. Thus aggregate source-recovery means are comparatively stable in this sample, while the identity of workload-passing modules remains run-sensitive. The larger differences from A cannot be assigned solely to randomness because its recorded routes are not uniform. The main-paper intervals still quantify module sampling only, and small differences between agents should not be interpreted as stable rankings.

S6 Reference-Guided Readability Judge

DeepSeek-V4-Pro-0813 receives the anonymized candidate, clean reference, and protected input under neutral labels. At temperature zero, one judgment per output rates identifier naming, code style, AST structure, and recovery of meaningful decision structure from 1 to 10. Their unweighted sum ranges from 4 to 40. Each dimension cites a file, original line, and short excerpt. The prompt treats code as untrusted data and forbids execution, tool use, system inference, and inference of functional correctness.

Anchors map 1–2 to absent or opaque recovery, 3–4 to limited, 5–6 to partial, 7–8 to mostly recovered, and 9–10 to clear and faithful recovery. Raw responses, hashes, validation, and deterministic normalizations are retained. The same rubric is used for both corpora and protections.

Repeat stability. We measured run-to-run variation on 50 agent outputs and 20 fixed-pipeline outputs. Each group samples five outputs per system and protection (JSO/full and VM), using separate fixed seeds; the fixed-pipeline group includes webcrack and JSimplifier. Eligibility requires an archived rating, locally reproducible candidate, reference, and protected files, and a serialized request below 3 MB. Each output received three fresh calls with the same model, prompt, temperature, and pinned provider endpoint, yielding 210 validated ratings. Calls that failed transport, response-schema, or evidence-citation validation were retried only for the affected output and repeat; invalid responses were excluded.

Table S5 reports exact agreement, bounded disagreement, rank stability, and absolute agreement. $Exact_3$ is the fraction of outputs for which all three scores are identical, whereas $Exact_p$ aggregates the three pairwise comparisons per output. *Within* permits a one-point difference for a dimension and a four-point difference for the 4–40 total. The reported ρ is the mean pairwise Spearman correlation. ICC_1 and ICC_3 are two-way absolute-agreement coefficients for one rating and the mean of three ratings, respectively.

For agents, the three mean total scores are 31.78, 31.98, and 31.60; for fixed pipelines, they are 11.45, 11.90, and 11.65. Total-score exact agreement across all three calls is 66.0% for agent outputs and 55.0% for fixed-pipeline outputs, while 100.0% and 96.7%, respectively, of pairwise totals differ by at most four points. The fixed-pipeline sample has greater within-output variation (mean SD

Table S5. Repeat stability of the reference-guided readability judge on 50 agent and 20 fixed-pipeline outputs, each rated three times. Agreement columns are percentages; $\overline{SD}$ is the mean within-output standard deviation. Groups are summarized separately.

Score	Exact ₃	Exact _p	Within	$\overline{SD}$	ρ	ICC ₁	ICC ₃
Agent outputs ($n = 50$)							
Identifier naming	94.0	96.0	100.0	0.035	0.907	0.994	0.998
Code style	84.0	89.3	100.0	0.092	0.862	0.983	0.994
AST structure	86.0	90.7	100.0	0.081	0.966	0.985	0.995
Decision structure	80.0	86.7	100.0	0.115	0.958	0.980	0.993
Total	66.0	76.0	100.0	0.317	0.966	0.992	0.997
Fixed-pipeline outputs ($n = 20$)							
Identifier naming	100.0	100.0	100.0	0.000	1.000	1.000	1.000
Code style	70.0	78.3	95.0	0.252	0.938	0.860	0.948
AST structure	75.0	81.7	96.7	0.220	0.961	0.943	0.980
Decision structure	65.0	76.7	96.7	0.260	0.954	0.945	0.981
Total	55.0	66.7	96.7	0.670	0.979	0.957	0.985

0.670 versus 0.317). Its identifier-naming score agrees exactly across repeats but averages only 1.95, so this agreement is partly susceptible to a floor effect. These are separate, small stratified samples, not a test of between-class rating accuracy. The judge has not been calibrated against blinded human raters; readability results remain exploratory and reference-guided.

S7 Identifier Recovery and Input-Length Sensitivity

S7.1 Hidden-Identifier Recovery Audit

As a contamination diagnostic, we extract JavaScript binding names from each original and remove any whose exact identifier text occurs anywhere in its protected input. For each candidate, recovery requires that the candidate declare the same remaining name; textual mentions alone do not count. Table S6 reports the micro-average over all such original binding instances. The denominator is 5,700 instances under JSO/full and 6,081 under VM for every system. This conservative lexical test cannot distinguish memorization or retrieval from a model independently choosing a conventional name after decoding the program, and is therefore not evidence of contamination by itself.

S7.2 Protected-Input Length and Outcomes

Figure S1 reports the input-length sensitivity analysis. Within each protection condition, we sort the canonical protected builds by byte length and partition them into equally sized short, medium, and long groups. The groups are formed once per protection and then held fixed across the nine model-based systems. Readability means omit only unavailable ratings. Traditional tools are excluded from this analysis because protected-input length is examined here as a model-facing constraint; their complete automatic outcomes remain in the main paper.

The sensitivity cohort spans 13–397 kB under JSO/full (median 52 kB) and 228–322 kB under VM protection (median 242 kB). In the raw group summaries, the long-input group has lower Execution correctness than the short group for all five agents under both protections except OpenHands under JSO/full, whose rate changes from 92% to 88%. The declines are largest for Claude Code and OpenCode under JSO/full (88% to 36% and 92% to 40%) and for OpenCode under VM (60% to 4%). CodeBLEU-R and ROUGE-L also decline from the short to the long group for nearly every model

Table S6. Verbatim recovery of original binding names absent from the protected input. Values are micro-averaged percentages over binding instances.

System	JSO/full	VM
webcrack	0.0	0.0
JSimplifier	0.8	1.8
Static GPT-sol	16.0	11.5
Static GLM-5.2	30.5	11.5
Static DeepSeek-V4-Pro-0813	27.6	10.3
Static Kimi-K2.5	17.3	6.8
OpenHands	25.8	16.8
Claude Code	20.9	16.4
OpenCode	12.3	15.5
Kimi Code	18.4	16.4
Codex	20.0	17.5

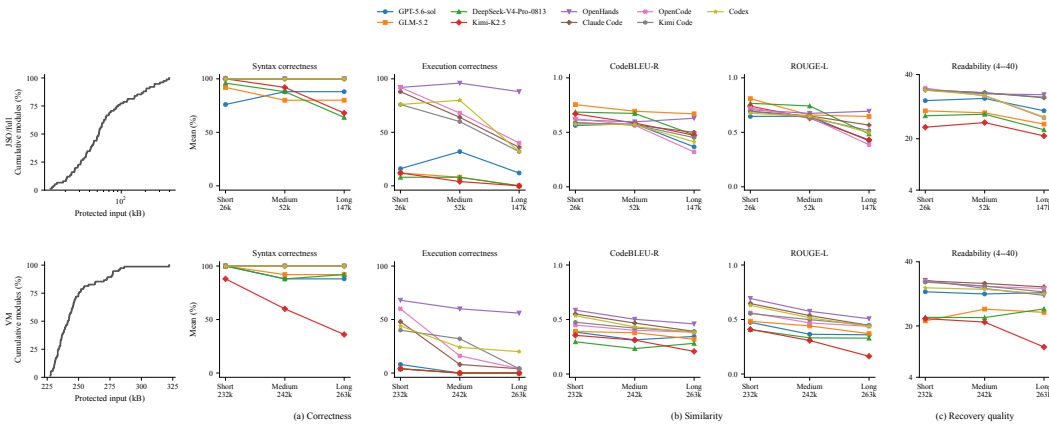


Fig. S1. Protected-input length and deobfuscation outcomes. Rows separate JSO/full and VM protection. The first column shows the canonical protected-build length distribution; the remaining columns report group means for the three evaluation levels. Short, medium, and long groups are defined independently within each protection. Readability uses the available DeepSeek-V4-Pro-0813 ratings. No background grid is used.

system; JSO/full OpenHands is the only exception for CodeBLEU-R. Readability declines for all nine systems under JSO/full and for seven of nine under VM.

These grouped patterns are descriptive. Canonical protected length is strongly correlated with original-bundle size (Spearman $\rho = 0.954$ for JSO/full and 0.953 for VM). After rank-adjusting for original size and applying 5,000 project-cluster bootstrap draws, every JSO/full Execution, CodeBLEU-R, ROUGE-L, and Readability interval includes zero. Under VM, the adjusted negative association with Execution excludes zero for Static DeepSeek, Claude Code, and OpenCode, while most other intervals include zero. Strong collinearity prevents the analysis from separating protection expansion from underlying program size. Figure S1 therefore supports a length-related sensitivity finding rather than a causal claim or a replacement for the 93-module main analysis.

S8 Cost Accounting and Coverage

Cumulative recorded input adds uncached input, cache reads, and cache writes only when reported separately; inclusive totals are not added twice. Output counts are generated model tokens. Counts describe provider-specific accounting, not prices. Runtime covers generation or tool execution and excludes offline scoring and judging.

Across the recorded runs, mean agent input spans 1.494–8.107 million tokens per system–module run, a 5.4-fold range, and mean output spans 15.8–47.7 thousand, a 3.0-fold range. Agent cache reads account for 93%–98% of recorded input. Across all 93 VM modules, median runtime spans 306.9–1,238.8 seconds, a 4.0-fold range.

These resource orderings do not match VM workload success: Kimi Code and OpenCode both pass 24 modules despite 4.4-fold different mean inputs and 2.3-fold different median runtimes, while Codex uses the least recorded input and passes 25. OpenHands passes 54 but records the longest median runtime. RQ4 retains one recorded run per system–subject cell; the targeted study in Supplement S5 assesses output stability and does not estimate run-to-run cost variation.

References

- [1] Guoqiang Chen, Xin Jin, and Zhiqiang Lin. 2025. JsDeObsBench: Measuring and Benchmarking LLMs for JavaScript Deobfuscation. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*. ACM, 36–50. <https://doi.org/10.1145/3719027.3744871>
- [2] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. <https://aclanthology.org/W04-1013/>
- [3] Obfuscator.io. 2026. Direct eval() Disables VM Obfuscation. <https://obfuscator.io/docs/vm-obfuscation/direct-eval>. Accessed September 2026.
- [4] Obfuscator.io. 2026. How VM Transforms Code. <https://obfuscator.io/docs/vm-obfuscation/how-it-works>. Accessed September 2026.
- [5] Obfuscator.io. 2026. Options Reference: vmObfuscation. <https://obfuscator.io/docs/options>. Accessed September 2026.
- [6] Obfuscator.io. 2026. Using the NPM Package. <https://obfuscator.io/docs/vm-obfuscation/npm-package>. Accessed September 2026.
- [7] Obfuscator.io. 2026. VM Debug Protection. <https://obfuscator.io/docs/vm-obfuscation/debug-protection>. Accessed September 2026.
- [8] Obfuscator.io. 2026. VM Obfuscation. <https://obfuscator.io/docs/vm-obfuscation>. Accessed September 2026.
- [9] Obfuscator.io. 2026. VM Self Defending. <https://obfuscator.io/docs/vm-obfuscation/self-defending>. Accessed September 2026.
- [10] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A Method for Automatic Evaluation of Code Synthesis. *arXiv preprint arXiv:2009.10297* (2020).